Activity: Residential Energy Challenge

You are required to create a neural network model to **predict residential electricity consumption for the next 24 hours at hourly intervals** for your household assigned by the course platform. The platform progressively reveals real data as time passes, and your model will be queried once per day to produce one complete 24-hour forecast. Previous exploratory analysis of the dataset, i.e., identifying missing values, visualising seasonal patterns, inspecting weekday vs. weekend behaviour, correlation between variables, is strongly encouraged before designing the model.

Note	All feature scaling, de-scaling, and preprocessing must live **inside the exported ONNX model**. The platform always sends raw values (kWh and timestamp) and expects the output already in real kWh. **Do not change the ONNX contract**.

You may experiment freely with different architectures (MLP, CNN 1D, LSTM, GRU, Transformer), training strategies, loss functions, optimizers, and train/validation splits. However, the ONNX input/output contract (described below) is **non-negotiable** and must be preserved exactly. Any model that breaks the contract will be rejected by the platform.

Dataset

The historical consumption data for your assigned household must be downloaded from the Residential Energy Challenge platform using the **Download historical data** button. The file is delivered already named `raw.csv` and must be placed inside the project folder at:

```
data/raw.csv
```

The dataset contains two columns: `timestamp` (hour in platform local time, continuous and strictly hourly) and `kwh` (net energy value for that hour, in kWh). Sign convention: a positive value is a net import from the grid, and a negative value is a net export to the grid. Each download includes only the data available at the moment of the download and ends at the last completed day. You may re-download as often as you like to refresh your training set.

Workflow

Follow these steps, in order:

1. **Environment.** Create a Python virtual environment inside the project folder and install the dependencies with `pip install -r requirements.txt`.
2. **Download your data.** Open the platform, locate your assigned household and press “Download historical data”. Save or move the downloaded file so its final path is `data/raw.csv`.

3. **Verify the dataset.** Run `python starter/dataset.py`. You should see the shapes `(168, 3)` and `(24,)` printed, confirming that windows are being built correctly. The script also validates column names, timestamp continuity, duplicates and missing values.
4. **Edit the architecture.** The default `starter/model.py` is an intentionally simple MLP baseline. You are expected to redesign it. Preserve the `[batch, 168, 3] → [batch, 24]` contract and keep all preprocessing inside the model.
5. **Train the model.** Run `python starter/train.py`. The best checkpoint is saved as `starter/model.pt`. You may adjust epochs, batch size, learning rate, loss, optimiser, scheduler, early stopping and the train/validation split in `dataset.py`.
6. **Evaluate locally.** Run `python starter/evaluate.py`. The script reports MAE and MSE on your validation set and compares them against a persistence baseline (same 24 hours of the previous day). A useful model must beat persistence consistently. A positive MAE skill means your model is better than doing nothing.
7. **Export to ONNX.** Run `python starter/export.py`. This generates one self-contained `starter/model.onnx`. If a `model.onnx.data` file appears, the export failed. You must fix it before continuing.
8. **Validate before uploading.** Run `python starter/validate_onnx.py`. This script runs locally the exact same checks that the server will run on upload. Do not upload if any check fails.
9. **Upload to the platform.** Use the **Upload ONNX model** button in the platform. Only `starter/model.onnx` is accepted. The upload automatically supersedes any previous submission from your group.

Submission requirement	Upload only `starter/model.onnx` (one single file, maximum 25 MB). Final submission deadline: **December 15, 2025 at 23:59**. The evaluation window is **December 15, 2025 to January 15, 2026**, during which no new submissions can be made. Written report: maximum 8 pages (excluding code), Arial 12, no space between paragraphs.

Model contract

The platform accepts only ONNX models that respect the following interface exactly. The input tensor must be named `input`, of dtype `float32` and shape `[batch, 168, 3]`, where channel 0 is the raw kWh value, channel 1 is the hour of day as a value between 0 and 23, and channel 2 is the day of week as a value between 0 (Monday) and 6 (Sunday). The output tensor must be named `output`, of dtype `float32` and shape `[batch, 24]`, with the predicted values already expressed in real kWh. The first axis (`batch`) is dynamic, so the same model can be queried with any number of windows at once.

The starter model demonstrates the correct approach. During training, `mu` and `sigma` are computed from the training set and passed to the model as PyTorch buffers. Inside `model.forward()`, the kWh channel is standardised, the hour channel is divided by 23, the day-of-week channel by 6, and the final output is de-normalised back to real kWh. **You do not need to standardise anything manually**, the model handles it automatically both at training time and at inference time, and the scaling parameters travel baked into the ONNX file.

Evaluation

After each upload, your model is activated and all previous submissions from your group are marked as superseded (their metrics stay visible in your group dashboard but no longer update). Every day at **00:00**, the platform performs two operations automatically:

- It feeds your active model with the 168 hours of real consumption preceding midnight and stores its **forecast for the next 24 hours**.
- It closes the previous day by computing **MAE and MSE** of the forecast made 24 hours earlier against the real values that have now been fully revealed, and compares them against a persistence baseline (same 24 hours of the day before).

The leaderboard displays the **mean daily MAE** of each active submission, with a minimum of 7 evaluated days to appear in the ranking. Submissions below that threshold appear in a separate "evaluating" section.

The **final grade** is based exclusively on the evaluation window December 15 to January 15, during which every active model is evaluated under identical conditions (same days, same hours, same ground truth). All metrics shown earlier in the course are purely informative and serve as motivation to iterate.

Questions to answer:

- Which neural network architecture did you choose, and why? Describe its layers and total parameter count.
- How did you split your historical data into training and validation? Why is this split appropriate for a time-series forecasting problem?
- Did you detect any missing values, anomalies, or clear seasonal patterns in your household's consumption? How did you handle them?
- Which loss function and which optimiser did you use? Did you try alternatives, and if so, how did they compare?
- Report your validation MAE, MSE and the MAE skill against the persistence baseline. Was your model consistently better than persistence? On which days did it struggle the most?

- Describe at least one experiment where you varied one single factor (hidden size, number of layers, learning rate, dropout, data augmentation, etc.) and discuss its effect on validation metrics.
- How did your model behave on the platform leaderboard? Did the live daily metrics match your local validation results?
- Attach your final modified files: model.py, train.py, and dataset.py.